

Evaluating Non-Generative Decision Models in Sequential Gridworld Tasks

Mateus Pereira
Independent researcher

Abstract

Can models that choose from supplied actions guide an agent all the way to a goal? System-One Control Bench tests this across 100 symbolic gridworld puzzles, ten input conditions and five action spaces. An exact solver identifies every optimal move, including ties. With full context, DeepSeek V4.1 Flash (reasoning) has the highest success rate among the models, 80%; Jev completes 57% of games. Adding simulated action outcomes to the map improves Jev’s success rate by 16 percentage points. Longer sequences of actions do not consistently help. With full context, Jev is 90% optimal on an exam of 495 scripted positions shared by all models and 79% optimal in its own games after equal puzzle weighting, with 57% game success. Strong performance on scripted positions does not establish reliable navigation.

1 Introduction

A model may choose a sensible action when shown a single situation, yet struggle when its own decisions determine what happens next. We test this in a small gridworld where every optimal move can be computed exactly.

A non-generative decision model returns probabilities over supplied options. In sequential control, those choices must guide the agent all the way to the goal. Jev’s use in chess and software control motivates this test [1], [2], [3]. The map contains all information needed to navigate. We vary the input descriptions and the actions available between observations.

“System-One” refers to fast decisions without deliberate reasoning, after the System 1 / System 2 distinction. We evaluate Laya’s Typed-Decisions checkpoint, called Laya below; its decision interface is `system_one`.

Five research questions guide the report:

1. **RQ1:** How often do models complete games?
2. **RQ2:** Which added input descriptions help?
3. **RQ3:** Do good decisions on an exam of fixed positions, shared by all models, translate into completed games?
4. **RQ4:** What changes when actions contain sequences of steps?
5. **RQ5:** How informative are the returned probabilities, and at what cost and latency?

We provide exact labels that accept every equally optimal move, and tests of how input descriptions and available actions affect performance. Raw trajectories, replay validation and reproducible analysis accompany the results. Solver verification is established in prior planning evaluation [4]. Here it lets us inspect decisions by non-generative models.

2 Related work

Jev has been evaluated as a legal-move selector in chess [1] and compared with Laya and a local Qwen3 decision model [5]. Laya’s evaluated checkpoint specializes in four workflow domains and cautions about transfer [6]. GLiClass performs zero-shot classification over supplied labels in a single forward pass [7]; treating option texts as labels lets us evaluate such a classifier as a sequential controller.

PlanBench tests planning and reasoning about change in formal domains [4]. AgentBench covers eight interactive environments [8], and BALROG evaluates agents across games of differing difficulty [9]. We instead vary input components within one small environment to diagnose non-generative

decision models. BabyAI [10] and MiniGrid [11] provide related gridworld tasks; compass moves here separate map interpretation from orientation control.

SayCan combines language-based skill selection with feasibility estimates [12], while SayCanPay adds learned domain knowledge [13]. Our simulated outcomes are deterministic transitions supplied by environment code. Multiple-choice selection is sensitive to option identifiers and order [14], motivating seeded shuffling without assuming it removes bias. Calibration methods distinguish confidence from correctness [15]; our target accepts any solver-optimal move, rather than a unique class label.

3 Benchmark design

A puzzle contains walls (#), floor (.), a moving piece (A), goal (G), and optionally a key (K) and locked door (D). Compass steps move north, south, east or west. Collecting the key allows passage through the door; a wall or locked door otherwise blocks movement. Reaching the goal wins. Breadth-first search over position, inventory and door state labels every move that begins a shortest route, preserving ties.

```

0123456789  Map alone: position (7, 1), empty inventory, map and object coordinates;
0 #####          "What is the best next move?"
1 #.....AX#  Simulated outcomes: "move south (down): you move to (7, 2) and pick up
2 #...##*K.#          the key".
3 #D****#...#  Shortest route: south, west, west, south, west, west, west, south.
4 #G#.#...##
5 #####

```

Figure 1: The frozen example position. Stars trace one shortest route through K and D to G; X marks east, a suboptimal move toward open floor. Stars and X replace floor only in this figure. Appendix A gives its full-context request.

The 100 puzzles span levels 1, 2, 3, 4, 5, 6, 8, 10, 12, 15 and 20, where level is shortest distance in compass steps. Levels 1 and 2 contain five puzzles each; others ten. Seeded generation creates rooms and mazes, with one hand-made level-10 maze. Higher levels add mandatory detours and key-door dependencies; the wall-aware greedy baseline fails on all 40 puzzles at levels 10 and above. Extra outer-wall rings in half of the puzzles at levels 12, 15 and 20 add irrelevant map text without changing the interior.

A **move** is one model decision; a **step** is a compass displacement within it. Action spaces offer one step (compass, 4 options), exactly two (two-moves, 16), up to two (up-to-two-moves, 20), exactly three (three-moves, 64), or up to three (up-to-three-moves, 84). All sequences are offered. A blocked step is wasted and execution continues; reaching the goal immediately ends the move. The next observation follows execution. Changing the formulation changes option count, text length and steps executed before the next observation; several sequences can end at the same position. These experiments compare the whole interface rather than option count alone.

Every request includes rules, the numbered map, position, inventory and object coordinates. A **local state description** explains adjacent cells and relative object locations. **Interaction history** lists prior moves and their results. **Simulated action outcomes** tell the model what would happen after each option. An **explicit subgoal** names the next target. The local descriptions and simulations are computed from information already present in the map. History is not needed to reconstruct the state, but may expose loops. The ten conditions add each component to the map alone or remove it from full context (Table 1).

Table 1: Condition-ID mapping. Full context includes all four components.

Artifact ID	Paper name
map	Map alone
map+surroundings	Map with local state description
map+memory	Map with interaction history
map+lookahead	Map with simulated action outcomes
map+subgoal	Map with explicit subgoal
everything	Full context
everything-surroundings	Full context without local state description
everything-memory	Full context without interaction history

Artifact ID	Paper name
everything-lookahead	Full context without simulated action outcomes
everything-subgoal	Full context without explicit subgoal

4 Evaluation protocol

4.1 Models and baselines

We compare three kinds of player: non-generative decision models (Jev, Laya Typed-Decisions and GLiClass); a generative model used through option scoring, which generates nothing here (Qwen3.5-4B [16]); and chat models that generate an option ID (Gemma 4 26B [17] and DeepSeek V4.1 Flash [18]).

Random chooses uniformly from the offered actions. Greedy moves toward the next target, and greedy-walls also avoids walls. The solver always chooses an optimal move. The greedy baselines use compass moves only. Each model sees the request without direct access to the environment or solver.

Jev (jev-1.13.0) chooses the option with the highest returned probability. GLiClass scores the option texts independently, using the state as classification text and the question as the task. The controller normalises these scores to sum to 1. Qwen3.5-4B scores option numbers in its chat template and selects the most likely one.

Laya receives the state, question and option texts, with option IDs omitted. Requests above 8,192 tokens or options above 48 tokens are rejected rather than truncated. The Typed-Decisions checkpoint was trained for four workflow domains and warns about transfer and large option sets [6].

Gemma and DeepSeek use OpenRouter with DeepInfra as the fixed host and JSON-schema output constrained to an option ID. Direct answers have reasoning off. DeepSeek V4.1 Flash (reasoning) permits 1,024 reasoning tokens. Every model has compass results; sequence-rule results exist for Jev, Gemma 4 26B, Laya and GLiClass only (Qwen3.5-4B and both DeepSeek configurations ran compass moves only).

A missing, invalid or capped answer ends the game as an error and counts as a loss. All 242 evaluated combinations of player, action rules and input condition have zero errors in the retained records. Transient failures occurred; errored games were replayed with `--resume`. Option order is shuffled with a seed from the puzzle and move number. Appendix A.1 records prompt development and the later question correction; Appendix C gives the inference and replay checks.

4.2 Metrics and statistics

Success rate is the share of games reaching the goal within the move limit (the `won` field in the records). The limit is twice the solver’s fewest moves. Progress is one minus the closest compass-step distance reached divided by the starting distance. Final-state progress uses the ending position and can be negative. SPL is zero for a loss; for a win, it divides the fewest moves by the larger of that number and the moves used [19]. SPL and the limit count model decisions. Optimality minimises decisions and accepts every tied best move, including sequences with wasted steps.

Intervals use 10,000 puzzle resamples and the 95% percentile-bootstrap interval. They describe variation across the evaluated puzzle collection, not repeated calls. Component comparisons pair the same puzzles. McNemar’s exact test compares game completion; Holm correction covers eight component contrasts per model. Every evaluated model/input has 100 games.

We test whether the selected option’s probability predicts solver-optimality. This is an operational confidence diagnostic, not a complete test of the returned action distribution. Ten-bin ECE and binary Brier score use this target [15]. Two tied best moves at 0.5 each give an optimal chosen move a score of 0.5. GLiClass supplies normalised independent scores; Qwen supplies option-number token probabilities, so their origins differ.

The exam uses 495 scripted compass positions from 100 puzzles. Each model answers once per item under map alone and full context (990 answers). Prefixes use the ordinary game, preserving history and option shuffles. Errors are incorrect; any solver-best move is optimal. Rates average within puzzles, then equally over puzzles. Appendix C gives item counts and construction details.

5 Results

5.1 Success rate and efficiency

DeepSeek V4.1 Flash (reasoning) wins more games than any other model, with non-overlapping intervals under both inputs. Its success rate is 80% [72, 87] with full context and 67% [57, 76] with the map alone. Each estimate uses 100 games.

Jev wins 32% with the map alone, below the wall-aware greedy baseline’s observed 37% and above random’s 4%. With full context, Jev, Gemma and DeepSeek without reasoning win 57%, 61% and 60%. Paired success-rate differences for Jev minus Gemma are -3 percentage points $[-11, 5]$ with the map and -4 $[-10, 2]$ with full context; against DeepSeek they are -7 $[-14, 0]$ and -3 $[-11, 5]$. These intervals establish no advantage for Jev, without showing equality or inferiority. Laya and GLiClass win 2% and 3% with the map, near random; full context raises them to 12% and 13%. DeepSeek V4.1 Flash (reasoning) wins 80% with full context and 67% with the map alone.

Table 2: Compass success rate and efficiency. Each model/input has 100 games; brackets give 95% intervals that resample puzzles; baselines use the map alone. Full means full context.

Model/baseline	Map: success	Full: success	Map: progress/SPL	Full: progress/SPL
Solver	100% [100, 100]	—	1.000 / 1.000	—
Greedy (walls)	37% [28, 46]	—	0.475 / 0.370	—
Greedy	28% [20, 37]	—	0.356 / 0.280	—
Random	4% [1, 8]	—	0.249 / 0.033	—
Jev	32% [23, 41]	57% [47, 66]	0.425 / 0.319	0.683 / 0.541
Laya	2% [0, 5]	12% [6, 19]	0.105 / 0.020	0.264 / 0.105
GLiClass	3% [0, 7]	13% [7, 20]	0.104 / 0.030	0.216 / 0.130
Gemma 4 26B	35% [26, 44]	61% [51, 70]	0.452 / 0.322	0.714 / 0.568
Qwen3.5-4B	14% [8, 21]	45% [35, 55]	0.335 / 0.124	0.661 / 0.387
DeepSeek V4.1 Flash	39% [30, 49]	60% [50, 69]	0.545 / 0.347	0.715 / 0.542
DeepSeek V4.1 Flash (reasoning)	67% [57, 76]	80% [72, 87]	0.787 / 0.618	0.864 / 0.741

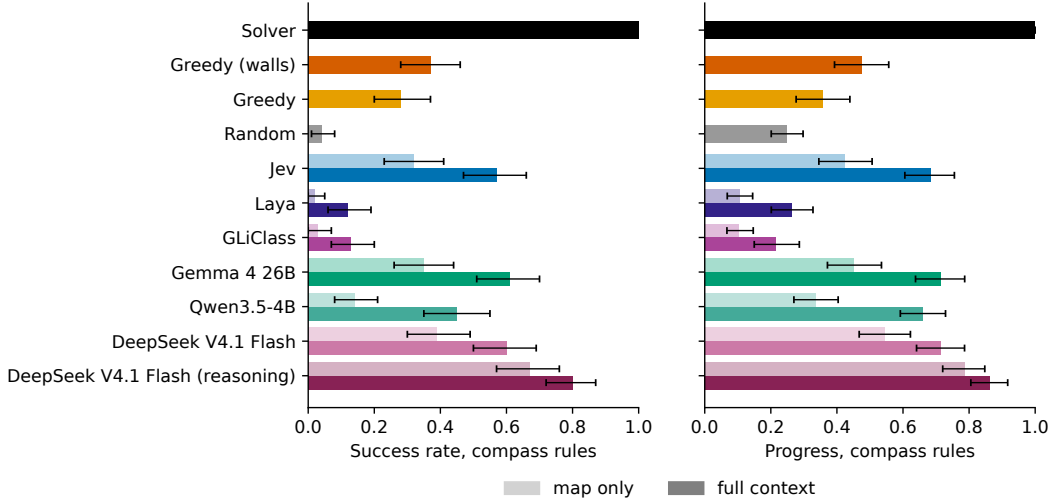


Figure 2: Compass success rate and progress with the map alone (pale) and full context (darker). Intervals resample puzzles. Each model/input uses 100 games.

5.2 Effects of input components

We compare the same puzzles with and without each component. For Jev, adding simulated outcomes to the map raises success rate by 16 percentage points [9, 23] and progress by 0.18 [0.13, 0.24]. Adding interaction history raises progress by 0.15 [0.11, 0.21]. Both additions pass the Holm-adjusted success-rate test. These comparisons measure improvements over the map alone; they do not establish that simulations outperform history.

The effect depends on what else the request contains. Including history in full context adds 0.06 [0.01, 0.10] progress, whereas including simulated outcomes changes progress by 0.00 $[-0.04, 0.05]$.

Neither corresponding success-rate contrast passes Holm correction. Several descriptions may help with the same decision.

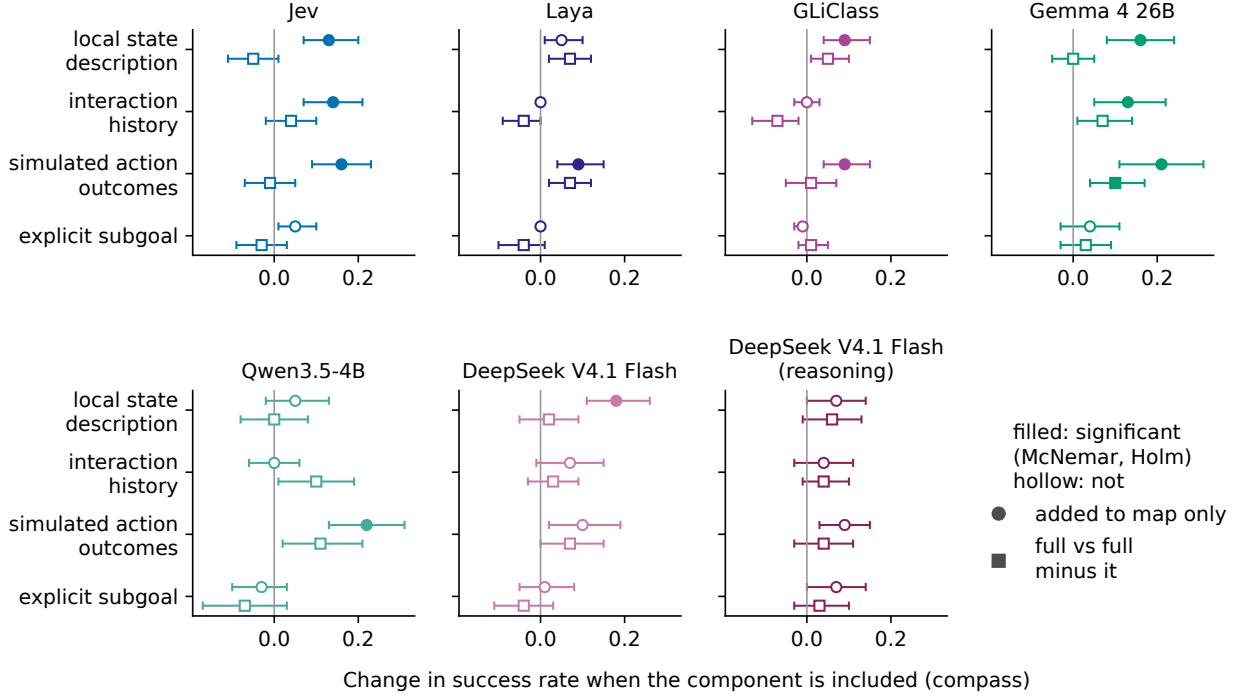


Figure 3: Paired change in success rate when a component is included. Circles compare adding it to the map alone; squares compare full context with and without it. Positive means inclusion helps. Filled markers identify McNemar tests that pass Holm correction.

5.3 Exam

On the exam, DeepSeek V4.1 Flash (reasoning) reaches 94% [91, 96] optimality with full context. Jev and Gemma both reach 90% [87, 93]; their paired difference is 0 percentage points [−1, 2]. DeepSeek without reasoning reaches 86% [83, 90], with a Jev-minus-DeepSeek paired difference of 4 points [1, 6]. Full context improves optimality for every model: 16 points for Jev, 21 for Laya and GLiClass, 20 for Gemma, 30 for Qwen3.5-4B, 19 for DeepSeek without reasoning and 5 for DeepSeek V4.1 Flash (reasoning). All paired input-effect intervals exclude zero.

With full context, Jev is 90% optimal on shared exam positions, 79% optimal in its own games after equal puzzle weighting, and has 57% game success. With the map alone, its exam optimality is 74% and its equally weighted game optimality is 38%. Accuracy across all moves made during the model’s games is 16% with the map and 64% with full context. Weighting explains part of the gap. Jev’s full-context start and late-route exam scores are 86% and 97%; the positions differ in more than history.

5.4 Blocked moves and progress

With the map alone, 82% of Jev’s moves leave the board unchanged. Without history, a blocked move leaves the board and its description unchanged, and only the option order changes, so Jev can choose the same move into the same wall again. With full context, the unchanged-board share is 0.4%, yet 43% of games still fail. Final-state progress is 0.65, below progress of 0.68. The distinction matters more for Qwen3.5-4B: its full-context scores are 0.56 and 0.66. A game can approach the goal and then finish farther away.

5.5 Success rate by level

At levels 12, 15, 20, Jev’s full-context success rates are 20%, 10%, 0%. Each level contains ten puzzles. DeepSeek V4.1 Flash (reasoning) wins 70%, 40% and 30% at those levels with full context, compared with 50%, 20% and 20% with the map alone. The higher levels remain difficult even with reasoning. These rates describe the generated levels.

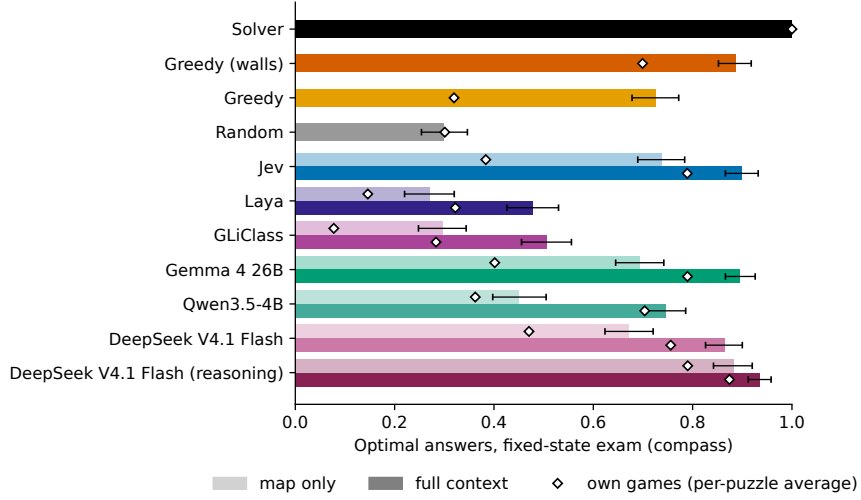


Figure 4: Optimality on the same 495 exam items. Pale and dark bars show map alone and full context; intervals resample puzzles. Diamonds show own-game optimality averaged within puzzles and then equally over puzzles, matching exam weighting. Baselines have one bar.

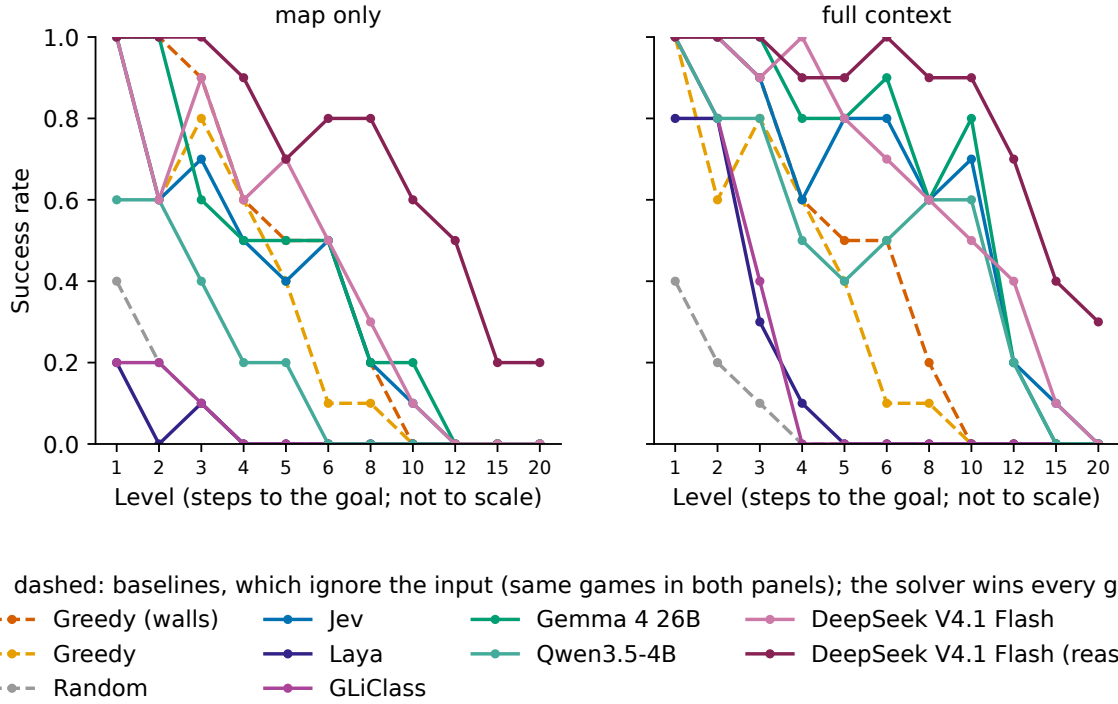


Figure 5: Compass success rate by categorical level with the map alone and full context. Labels give shortest-route distance in compass steps; horizontal gaps do not represent distance differences. Levels 1 and 2 have five puzzles each; others ten.

5.6 Action spaces

For Jev with the map alone, success rate is 32% under compass, 40% under two-moves, 47% under three-moves and 26% under up-to-three-moves. With full context, compass and up-to-three-moves give 57% and 49%; Gemma wins 61% under both. Longer sequences do not consistently improve success rate. Option count and text length also change.

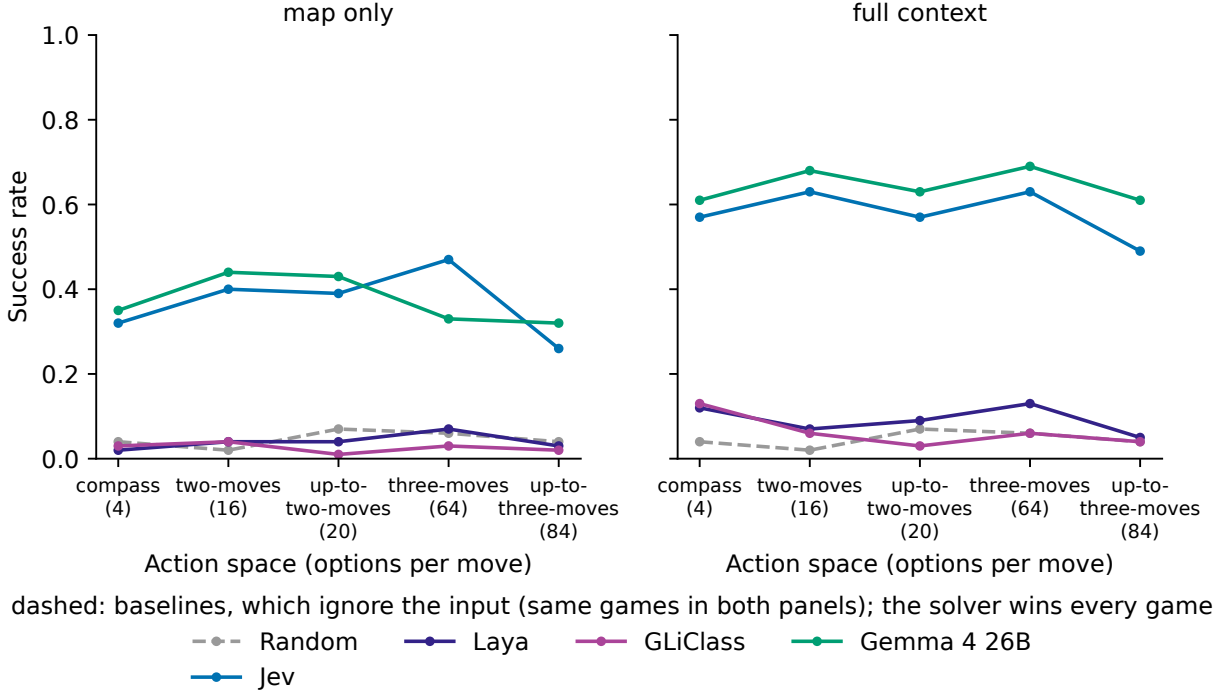


Figure 6: Success rate across compass, two-moves, up-to-two-moves, three-moves and up-to-three-moves (4, 16, 20, 64 and 84 options). Step count and observation interval also change. Qwen3.5-4B and both DeepSeek configurations ran compass moves only. Random is the same in both panels.

5.7 Probability analysis

Across compass conditions, Jev’s selected-option probabilities exceed its observed optimality on its own trajectories: mean probability is 0.719, while 51% of chosen moves are optimal. Its ECE is 0.209 and binary Brier score 0.283 over 14,642 moves. A predictor that always reports the observed optimal-move frequency scores 0.250, an in-sample reference. Laya’s ECE is 0.133 with 17.5% optimality; Qwen3.5-4B’s is 0.064 with 47.5% optimality. Low calibration error does not establish effective navigation. These scores test whether selected-option probability predicts solver-optimality on each model’s own trajectories across ten inputs. They are an operational confidence diagnostic, not a complete test of the action distribution. On common exam questions, Jev’s ECE is 0.063 with the map and 0.059 with full context.

TypeSafe defines Choice confidence as $c = \frac{p_{\max} - \frac{1}{n}}{1 - \frac{1}{n}}$ [20]. Across Jev’s 14,642 compass moves, the maximum discrepancy from this formula is 0.023 and remains unexplained. For a fixed option count, the formula ranks moves exactly as the top probability does; it defines a rescaled score, rather than an independent correctness prediction.

5.8 Cost and latency

These estimates pool all compass conditions. Median latency measures the successful answering call alone, excluding retries, waits and model loading: 0.503 seconds for Jev, 0.653 for Gemma, 0.805 for DeepSeek without reasoning and 6.817 for DeepSeek V4.1 Flash (reasoning). Recorded cost per 100 games is \$0.059, \$0.047 and \$0.348 for the three chat configurations, respectively. Dollar costs are unavailable for Jev and local models. Different services and hardware limit comparisons; Appendix C records the setup and accounting boundaries.

6 Discussion and limitations

With full context, Jev is 90% optimal on shared exam positions, 79% optimal in its own games after equal puzzle weighting, and completes 57% of games. Ten independent 90%-accurate decisions would all be right about 35% of the time. This is not a prediction: errors can be recovered, routes differ in length, and decisions are not independent. Games expose states reached by each model

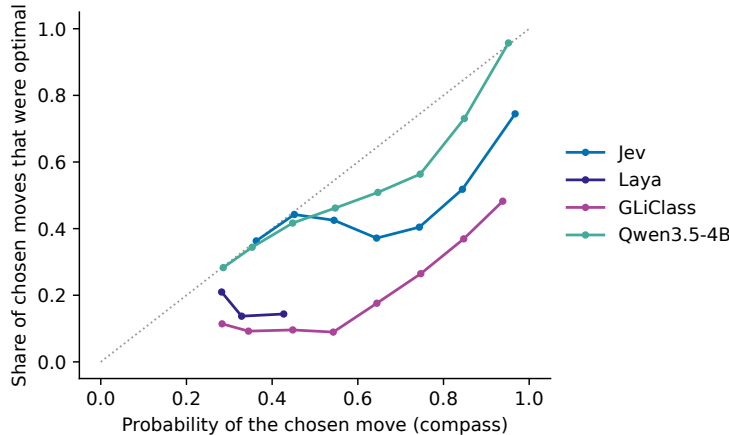


Figure 7: Reliability pooling all compass conditions; only bins with at least 30 moves are shown. Points compare mean chosen-move probability with optimality, accepting ties. The diagonal indicates agreement. Gemma and DeepSeek supply no option probabilities.

rather than scripted exam positions. Actions change later state distributions [21]; this is conceptual background, not a test of that method. We did not isolate the cause of the exam/game gap. Long failed games dominate pooled accuracy and probability diagnostics. Start and late-route exam positions also differ in distance and inventory, so their contrast cannot isolate history’s effect.

Full context nearly removes unchanged-board moves, yet 43% of games still fail. Avoiding blocked answers is only part of navigation. Explicit transitions may help, but wording and text length are not controlled separately. Comparisons that remove one component from full context do not show that any component is needed with the others present.

Sequence actions change option count, text length, observation frequency and outcome duplication. Fixed-length subgoal prompts say “shortest path”; up-to prompts say “the fewest turns.” Both are evaluated by decision count. Rounding also changes the attempted-step budget: a four-step puzzle allows four three-step moves (up to twelve steps), versus eight compass moves. These effects cannot be isolated. Levels vary in layout and route length; level effects do not isolate distance.

We test one symbolic world with five or ten puzzles per level, without a held-out set. Prompts preceded puzzles. Results are single trajectories: repeated calls changed Jev’s probabilities and chat answers; reruns would play different games (Appendix C). Provider changes, inference caps and input limits matter. Laya ran outside its training tasks. Hardware differs; missing costs limit price comparisons.

The environment, solver, frozen requests, trajectories, analysis and leaderboard are available at github.com/Exemocar/JevStuff. The repository documents submission and replay validation.

7 Conclusion

System-One Control Bench tests individual decisions and complete games. Full context improves Jev’s success rate and nearly eliminates its repeated blocked moves. Strong performance on these scripted exam positions does not establish reliable navigation; we did not isolate the cause of the gap. Longer sequences have mixed effects. These results concern the evaluated puzzles, inputs and inference settings.

AI use

This project was carried out with substantial help from AI assistants, chiefly Anthropic’s Claude models (through Claude Code) and OpenAI’s GPT models (through Codex). Under the author’s direction they wrote most of the code and the analysis scripts, drafted and revised the text of this report, and reviewed one another’s work. The author set the research question, made the design decisions, checked the results and reviewed the manuscript, and takes full responsibility for its contents.

References

- [1] M. Saplin, “TypeSafe Jev Played Chess — And Landed Next to Reasoning Models.” Accessed: Oct. 04, 2026. [Online]. Available: <https://dev.to/maximsaplin/typesafe-jev-played-chess-and-landed-next-to-reasoning-models-28ga>
- [2] droidrun, “mobile-jev: a standalone mobile agent for Android using Jev.” Accessed: Oct. 04, 2026. [Online]. Available: <https://github.com/droidrun/mobile-jev>
- [3] browser-use, “jev-ultrafast: a browser agent with an indexed action space.” Accessed: Oct. 04, 2026. [Online]. Available: <https://github.com/browser-use/jev-ultrafast>
- [4] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, “PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change,” in *NeurIPS Datasets and Benchmarks Track*, 2023.
- [5] P. Chopra, “Jev vs Laya vs our local Qwen3 decision model.” Accessed: Oct. 04, 2026. [Online]. Available: <https://gist.github.com/paraschopra/48c69b7edb99cd15137524379cceba3a>
- [6] Convai Innovations, “Laya Typed-Decisions: Model Card.” Accessed: Oct. 04, 2026. [Online]. Available: <https://huggingface.co/convaiinnovations/laya-typed-decisions>
- [7] I. Stepanov, M. Shtopko, D. Vodianytskyi, O. Lukashov, A. Yavorskyi, and M. Yaroshenko, “GLiClass: Generalist Lightweight Model for Sequence Classification Tasks,” *arXiv preprint arXiv:2508.07662*, 2025.
- [8] X. Liu *et al.*, “AgentBench: Evaluating LLMs as Agents,” in *ICLR*, 2024.
- [9] D. Paglieri *et al.*, “BALROG: Benchmarking Agentic LLM and VLM Reasoning On Games,” in *ICLR*, 2025.
- [10] M. Chevalier-Boisvert *et al.*, “BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning,” in *International Conference on Learning Representations*, 2019.
- [11] M. Chevalier-Boisvert *et al.*, “Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks,” *arXiv preprint arXiv:2306.13831*, 2023.
- [12] M. Ahn *et al.*, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [13] R. Hazra, P. Zuidberg Dos Martires, and L. De Raedt, “SayCanPay: Heuristic Planning with Large Language Models using Learnable Domain Knowledge,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 18, pp. 20123–20133, 2024, doi: [10.1609/aaai.v38i18.29991](https://doi.org/10.1609/aaai.v38i18.29991).
- [14] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, “Large Language Models Are Not Robust Multiple Choice Selectors,” in *ICLR*, 2024.
- [15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in *ICML*, in PMLR, vol. 70. 2017, pp. 1321–1330.
- [16] Qwen Team, “Qwen3.5: Towards Native Multimodal Agents,” *Qwen blog*, Feb. 2026, Accessed: Oct. 04, 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [17] Gemma Team, “Gemma 4 Technical Report,” *arXiv preprint arXiv:2607.02770*, 2026.
- [18] OpenRouter, “DeepSeek V4.1 Flash.” Accessed: Oct. 04, 2026. [Online]. Available: <https://openrouter.ai/deepseek/deepseek-v4.1-flash>
- [19] P. Anderson *et al.*, “On Evaluation of Embodied Navigation Agents,” *arXiv preprint arXiv:1807.06757*, 2018.
- [20] TypeSafe AI, “Confidence.” Accessed: Oct. 04, 2026. [Online]. Available: <https://docs.typesafe.ai/confidence>
- [21] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *International Conference on Artificial Intelligence and Statistics*, in PMLR, vol. 15. 2011, pp. 627–635.

A One complete example request

This is the compass, full-context request for Figure 1, with every field and all state text retained. Lines wrap only where needed to fit the page. All 50 frozen requests are in the repository’s `examples/` directory.

state:

You move one cell per turn: north (up), south (down), east (right) or west (left). Walls (#) block you, and walking into one wastes the move. Stepping onto the key (K) picks it up. A locked door (D) opens when you walk into it carrying the key, and you step through; without the key it blocks you like a wall. Reaching the goal (G) wins.

Map, with column numbers along the top and row numbers down the side (# = wall, . = floor, A = you, G = goal, K = key, D = locked door). A position (x, y) is column x, row y:

```
0123456789
0 #####
1 #.....A.#
2 #...#..K.#
3 #D...#..#
4 #G#.#...##
5 #####
```

You are A at (7, 1). You are carrying nothing.

Objects: goal G at (1, 4); key K at (7, 2); locked door D at (1, 3).

North of you is a wall. South of you is the key. East of you is open floor. West of you is open floor. The goal is 6 west and 3 south of you. The key is 1 south of you. The locked door is 6 west and 2 south of you.

Moves so far:

1. east: you move to (7, 1)
2. north: blocked, you stay at (7, 1)

model: jev-1.13.0

questions.move.type: choice

questions.move.instructions: Your next target is the key K at (7, 2). Which move is the first step of the shortest path to it?

questions.move.criteria:

- option_1: move south (down): you move to (7, 2) and pick up the key
- option_2: move north (up): blocked, you stay at (7, 1)
- option_3: move west (left): you move to (6, 1)
- option_4: move east (right): you move to (8, 1)

A.1 Prompt history

Prompts were developed by inspecting a few games on one or two puzzles at a few levels before generating the benchmark puzzles. Compass text is unchanged since 4d9a877 (23 September 2026, 20:10), before Jev’s reported results in b90ab4b (22:01) and 808785f (23:48). Sequence rules entered in 4da2c7d (24 September, 00:36); their two-/three-step prompts were fixed in 5f135ed (11:43), before runs at 13:41 committed in 1144516 (14:12). Times are local, UTC+02:00. On 26 September the up-to-subgoal question was corrected to ask for the route taking the fewest turns, and every affected game was replayed by every model or baseline that had evaluated it. The analysis uses the corrected records.

B Hypotheses specified before the main evaluation

Table 3: Hypothesis texts specified before the main evaluation are retained verbatim, including “won rate.” H1 and H7 are partly supported. H2 defines its planning subset.

Hypothesis	Observed result	Criterion met
H1: under map only, Jev beats random but not greedy (walls), on won rate and progress	won vs random: +0.28 [+0.19, +0.37]; progress vs random: +0.18 [+0.09, +0.26]; won vs greedy-walls: -0.05 [-0.12, +0.02]; progress vs greedy-walls: -0.05 [-0.11, +0.01]; 'not greedy (walls)': no advantage shown, but one is not excluded	partly
H2: Jev's wins fall with the level; no condition wins half of the planning puzzles	success-vs-level slope -0.061 to -0.046 per level; best on the 63 planning puzzles: 0.40 (everything-surroundings); planning: the 63 puzzles greedy-walls loses (100 - 37)	yes
H3: simulated action outcomes help most (best map+X, worst everything-X, on progress)	map+X: lookahead 0.61, memory 0.58, surroundings 0.57, subgoal 0.46; everything-X lowest: memory 0.63	partly
H4: explicit subgoal is second among additions, and helps mostly from level 3 up	rank 4 of 4; progress gain +0.02 at levels 3+, +0.10 at 1-2	no
H5: interaction history alone adds almost nothing (within 0.05 progress of map)	progress change +0.15 [+0.11, +0.21]	no
H6: full context has the best progress of the ten conditions and wins at least 28/100	best progress: everything-subgoal (0.73); full context wins 57/100	partly
H7: Jev is overconfident, but its p(chosen) still ranks its moves	mean p(chosen) 0.72 vs optimal share 0.51; optimal at p >= 0.9: 0.74 (3895 moves), below: 0.42; the split ranks, but the reliability curve is not monotone	partly

C Reproducibility

Table 4: Evaluated model IDs and inference settings. Revisions cached on the laptop: Laya 55cf4c4, GLiClass 668dd0d, Qwen3.5-4B 851bf6e. Software dependencies are locked in `uv.lock`.

Model	Model ID / host	Precision and inference limits
Jev	jev-1.13.0 / TypeSafe	Remote precision unreported; highest option probability.
Laya	convaiinnovations/laya, subfolder typed-decisions (also published as convaiinnovations/laya-typed-decisions [6]) / local	Precision unrecorded; 8,192 request tokens, 48 per option.
GLiClass	knowledgator/gliclass-modern-large-v3.0 / local	Precision unrecorded; 8,192 tokens, up to 128 labels.
Qwen3.5-4B	Qwen/Qwen3.5-4B / local	bfloat16; thinking off; scored option numbers, no generated answer.
Gemma 4 26B	google/gemma-4-26b-a4b-it / OpenRouter, DeepInfra	Remote precision unreported; reasoning off, 64 answer tokens.
DeepSeek V4.1 Flash	deepseek/deepseek-v4.1-flash / OpenRouter, DeepInfra	Remote precision unreported; direct: 64 answer tokens; reasoning: 1,024 + 64 tokens.

The Laya and GLiClass compass games on 25 September ran on the CPU of a second computer, without a GPU; its exact model is not recorded. Every other local run (Laya and GLiClass sequence-rule games on 26 September and later, Qwen3.5-4B, and all exam runs) used an Acer Predator PHN16S-71 laptop: Intel Core Ultra 9 275HX (24 cores), 32 GB DDR5-6400, NVIDIA RTX 5070 Ti Laptop GPU (12 GB, driver 581.80), Windows 11.

Laptop software: Python 3.11.9, CUDA 13.0, torch 2.14.0+cu130, transformers 5.17.0 and gliclass 0.1.20. Runs span 23 September–3 October 2026; result filenames in `analysis/manifest.toml` identify dates. Other-machine software versions and local precision, except Qwen3.5-4B, were not recorded here.

Latency measures the successful answering call alone, excluding retries, waits and model loading. Recorded USD costs sum provider-reported answering-call costs and are scaled per 100 games; earlier failed attempts have no recorded cost. Jev and local-model dollar costs are unavailable. Host/device differences prevent a controlled hardware comparison.

Chat requests set neither temperature nor `top_p`; they use provider defaults. Direct answers allow 64 tokens. DeepSeek V4.1 Flash (reasoning), artifact ID `deepseek-v4.1-flash-think`, allows 1,024 reasoning tokens plus 64 answer tokens. OpenRouter retries refused calls four times, after 5, 15, 30 and 60 seconds. Its timeout is 120 seconds for direct answers and 600 seconds for reasoning. Jev retries refused calls twice, after one and two seconds, but does not retry a 120-second timeout.

Laya receives the full state as the first argument of `system_one`. The controller expands the total and question/option token budgets to fit the request. Option IDs are omitted because Laya would print them within each option’s 48-token allowance. GLiClass uses a multi-label pipeline, with each independent score between 0 and 1 before normalisation. Qwen3.5-4B scores allowed option numbers in its chat template with thinking off and generates no answer.

The exam uses seed 0, with five items per puzzle or four at level 1. Item counts are 100 start, 95 on-route, 144 off-route, 88 after-blocked and 68 late-route (after the key where present). Missing kinds are filled with labelled off-route or after-blocked items, sometimes reusing a board. Random selects a fixed shuffled slot; chance optimality is 0.27.

Replaying every recorded game with the code current at each run and with the released code gives byte-identical requests: state, question and every option. The exception is the documented up-to subgoal question in the five subgoal conditions (Appendix A.1). Rerunning GLiClass, Laya and Qwen3.5-4B at 529 recorded positions (176, 124 and 229, respectively) reproduced their probabilities exactly. This includes Laya with its newer Hugging Face revision 7b928d8.

Jev returned different probabilities for identical requests at ten positions, with a spread up to 0.14 across three calls. On replay it chose the same move at 91 of 92 positions. DeepSeek V4.1 Flash gave up to four different answers in five identical calls at provider defaults. These checks show that reruns of Jev or the chat models would play different games. The reported results are single trajectories. Puzzle-bootstrap intervals describe variation across the evaluated puzzle collection, not repeated calls. Zero errors describes the retained records after errored games were replayed with `--resume`; transient failures occurred.

This report describes release v1.0.0 of the repository (code, puzzles, results and `analysis/manifest.toml`).

D Full component effects

Each row reports both paired contrasts for one component. “Add” compares map plus the component with map alone. “Full” compares full context with full context minus that component. Positive values mean including the component helps. Intervals resample puzzles; Holm correction covers eight success-rate tests per model. Success-rate changes are proportions.

Table 5: Complete paired component effects. Brackets give unadjusted 95% intervals; p-values are Holm-adjusted. Add and Full both report the effect of including the component.

Component	Add: success change	Add: progress	Add: Holm p	Full: success change	Full: progress	Full: Holm p
Jev						
local state description	0.13 [0.07, 0.20]	0.14 [0.09, 0.20]	0.001	-0.05 [-0.11, 0.01]	-0.02 [-0.06, 0.02]	0.719
interaction history	0.14 [0.07, 0.21]	0.15 [0.11, 0.21]	<0.001	0.04 [-0.02, 0.10]	0.06 [0.01, 0.10]	1.000
simulated action outcomes	0.16 [0.09, 0.23]	0.18 [0.13, 0.24]	<0.001	-0.01 [-0.07, 0.05]	0.00 [-0.04, 0.05]	1.000
explicit subgoal	0.05 [0.01, 0.10]	0.03 [0.00, 0.07]	0.312	-0.03 [-0.09, 0.03]	-0.04 [-0.09, -0.00]	1.000
Laya						
local state description	0.05 [0.01, 0.10]	0.10 [0.06, 0.16]	0.312	0.07 [0.02, 0.12]	0.12 [0.06, 0.17]	0.109
interaction history	0.00 [0.00, 0.00]	-0.02 [-0.05, 0.00]	1.000	-0.04 [-0.09, 0.00]	0.02 [-0.03, 0.07]	0.875
simulated action outcomes	0.09 [0.04, 0.15]	0.08 [0.03, 0.14]	0.031	0.07 [0.02, 0.12]	0.09 [0.03, 0.14]	0.109
explicit subgoal	0.00 [0.00, 0.00]	-0.01 [-0.04, 0.01]	1.000	-0.04 [-0.10, 0.01]	-0.02 [-0.07, 0.02]	0.875
GLiClass						
local state description	0.09 [0.04, 0.15]	0.11 [0.05, 0.17]	0.031	0.05 [0.01, 0.10]	0.08 [0.04, 0.13]	0.312
interaction history	0.00 [-0.03, 0.03]	0.04 [0.01, 0.08]	1.000	-0.07 [-0.13, -0.02]	-0.09 [-0.13, -0.05]	0.234
simulated action outcomes	0.09 [0.04, 0.15]	0.12 [0.06, 0.18]	0.031	0.01 [-0.05, 0.07]	-0.05 [-0.12, 0.02]	1.000
explicit subgoal	-0.01 [-0.03, 0.00]	0.05 [0.01, 0.09]	1.000	0.01 [-0.02, 0.05]	0.01 [-0.02, 0.05]	1.000
Gemma 4 26B						
local state description	0.16 [0.08, 0.24]	0.18 [0.11, 0.25]	0.003	0.00 [-0.05, 0.05]	0.01 [-0.03, 0.06]	1.000
interaction history	0.13 [0.05, 0.22]	0.12 [0.06, 0.19]	0.038	0.07 [0.01, 0.14]	0.05 [0.00, 0.10]	0.262
simulated action outcomes	0.21 [0.11, 0.31]	0.22 [0.14, 0.30]	<0.001	0.10 [0.04, 0.17]	0.09 [0.04, 0.13]	0.038
explicit subgoal	0.04 [-0.03, 0.11]	0.03 [-0.03, 0.09]	1.000	0.03 [-0.03, 0.09]	-0.00 [-0.04, 0.04]	1.000
Qwen3.5-4B						
local state description	0.05 [-0.02, 0.13]	0.13 [0.06, 0.20]	1.000	0.00 [-0.08, 0.08]	0.02 [-0.03, 0.07]	1.000
interaction history	0.00 [-0.06, 0.06]	0.05 [-0.01, 0.11]	1.000	0.10 [0.01, 0.19]	0.07 [0.01, 0.14]	0.383
simulated action outcomes	0.22 [0.13, 0.31]	0.22 [0.15, 0.29]	<0.001	0.11 [0.02, 0.21]	0.10 [0.03, 0.17]	0.303
explicit subgoal	-0.03 [-0.10, 0.03]	-0.07 [-0.13, -0.01]	1.000	-0.07 [-0.17, 0.03]	-0.03 [-0.10, 0.05]	1.000
DeepSeek V4.1 Flash						
local state description	0.18 [0.11, 0.26]	0.16 [0.11, 0.23]	<0.001	0.02 [-0.05, 0.09]	-0.01 [-0.05, 0.04]	1.000
interaction history	0.07 [-0.01, 0.15]	0.09 [0.02, 0.16]	0.717	0.03 [-0.03, 0.09]	0.02 [-0.01, 0.06]	1.000
simulated action outcomes	0.10 [0.02, 0.19]	0.13 [0.06, 0.19]	0.290	0.07 [0.00, 0.15]	0.08 [0.02, 0.14]	0.711
explicit subgoal	0.01 [-0.05, 0.08]	-0.01 [-0.07, 0.04]	1.000	-0.04 [-0.11, 0.03]	-0.07 [-0.13, -0.01]	1.000
DeepSeek V4.1 Flash (reasoning)						
local state description	0.07 [0.00, 0.14]	0.05 [-0.00, 0.10]	0.646	0.06 [-0.01, 0.13]	0.03 [-0.02, 0.07]	0.730
interaction history	0.04 [-0.03, 0.11]	0.02 [-0.03, 0.08]	1.000	0.04 [-0.01, 0.10]	0.02 [-0.01, 0.06]	1.000
simulated action outcomes	0.09 [0.03, 0.15]	0.05 [0.01, 0.09]	0.094	0.04 [-0.03, 0.11]	0.02 [-0.03, 0.06]	1.000
explicit subgoal	0.07 [0.00, 0.14]	0.04 [-0.01, 0.09]	0.646	0.03 [-0.03, 0.10]	0.01 [-0.03, 0.06]	1.000